



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	10 / Tipe Data List

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Sifat-Sifat List

List merupakan salah satu struktur data fundamental dalam Python yang memiliki kemampuan untuk menampung sekumpulan nilai sekaligus dalam satu variabel. Dibandingkan dengan string yang hanya dapat menyimpan urutan karakter, list jauh lebih fleksibel karena dapat menampung berbagai jenis tipe data secara bersamaan, mulai dari karakter, bilangan bulat (integer), bilangan desimal (float), bahkan list lain di dalamnya yang dikenal sebagai list bersarang (nested list). Secara penulisan, list dibentuk menggunakan tanda kurung siku [] dengan setiap elemen dipisahkan oleh tanda koma.

```
nilai_ujian = [80,75,70,90,81,84,92,71,65,80,70]
nama_pahlawan = ['Sukarno', 'Diponegoro', 'Jend. Sudirman', 'Cut Nya Dhien']
nilai_campuran = ['Javascript', 20, 34.4, True]
list_dalam_list = [23, [22, 20], 45]
```

Perbedaan lain antarlis dan string terletak pada kemampuan modifikasinya. List bersifat *mutable*, artinya elemen di dalamnya dapat diganti, ditambah, atau dihapus secara langsung tanpa perlu membuat objek baru. Sebaliknya, string bersifat *immutable*, sehingga setiap operasi yang seolah-olah mengubah string sebenarnya hanya menghasilkan string baru, sedangkan string aslinya tidak ikut berubah.

```
# definisikan list berisi 4 nilai
data = [10,20,30,40]
# ubah nilai index ke-0 menjadi 50
data[0] = 50
# isinya sekarang: 50, 20, 30, 40
print(data)
```

```
# definisikan sebuah string
nama = 'Antonius Rachmat'
# ubah karakter index ke-0 menjadi Z
nama[0] = 'Z'
# tidak akan mencapai baris ini karena muncul error
# TypeError: 'str' object does not support item assignment
print(nama)
```

Dari sisi penggunaan memori pun keduanya berbeda. Ketika dua variabel string memiliki nilai yang sama, Python akan mengarahkan keduanya ke satu objek yang sama di memori sebagai bentuk efisiensi. Namun untuk list, meski dua variabel memiliki isi yang sama persis sama, Python tetap mengalokasikan ruang memori yang terpisah untuk masing-masing, sehingga keduanya dianggap sebagai objek yang berbeda dan tidak saling berbagi.

Mengakses dan Mengubah Isi List

Elemen dalam list dapat diakses menggunakan nomor indeks, dimulai dari angka 0 untuk elemen pertama. Indeks negatif juga bisa dipakai untuk mengakses elemen dari sisi belakang. Selain membaca, isi list juga dapat diperbarui langsung melalui indeksinya. Selain itu, Python mendukung teknik *slicing* untuk mengambil sebagian elemen berdasarkan rentang indeks tertentu, serta penggantian beberapa elemen sekaligus dalam satu operasi.

```
thislist = ["apple", "banana", "cherry"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])

thislist[1] = "jeruk"

print(thislist)

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist2[1:3] = ["blackcurrant", "watermelon"]
print(thislist2)
```

Fungsi-fungsi Untuk List

```
thislist = [1,2,3,4,5]
thislist2 = [6,7,8]

listbaru = thislist + thislist2
listbaru2 = thislist * 2

print(thislist)
print(thislist2)
print(listbaru)
print(listbaru2)
```

Dua list dapat digabungkan menjadi satu menggunakan operator +, sementara operator * digunakan untuk mengulang isi sebuah list sebanyak jumlah yang ditentukan.

Python juga menyediakan sejumlah method bawaan untuk memanipulasi list, antara lain:

- **append()**

Digunakan untuk menyisipkan satu elemen baru di posisi paling akhir list. Elemen yang ditambahkan diperlakukan sebagai satu kesatuan objek, bahkan jika elemen tersebut berupa list lain. Misalnya, `nama.append(['tejo', 'ujo'])` akan menambahkan ['tejo', 'ujo'] sebagai satu elemen tunggal di akhir list `nama`.

- **extend()**

Berfungsi menambahkan elemen ke dalam list, namun cara kerjanya berbeda dari `append()`. Method ini memecah setiap elemen dalam iterable yang diberikan dan menambahkannya satu per satu ke dalam list asal. Contohnya, `nama.extend(['tejo', 'ujo'])` akan menambahkan 'tejo' dan 'ujo' sebagai dua elemen terpisah.

- **sort()**

Digunakan untuk mengurutkan seluruh elemen dalam list secara langsung tanpa menghasilkan list baru. Setelah method ini dipanggil, urutan elemen dalam list akan berubah mengikuti urutan ascending atau alfabetikal.

- **pop()**

Berfungsi menghapus elemen dari list berdasarkan posisi indeksnya, sekaligus mengembalikan nilai elemen yang dihapus tersebut. Ini berguna ketika kita ingin mengetahui nilai elemen yang dibuang.

- **del**

Digunakan untuk menghapus elemen berdasarkan indeks, tetapi tidak mengembalikan nilai elemen yang dihapus.

- **remove()**

Menghapus elemen berdasarkan nilainya, bukan posisinya. Elemen pertama yang cocok dengan nilai yang diberikan akan dihapus dari list.

- **reverse()**

Membalik susunan seluruh elemen dalam list, sehingga elemen terakhir menjadi yang pertama dan sebaliknya.

Selain method-method di atas, Python juga menyediakan fitur bernama *list comprehension* yang memungkinkan pembuatan list baru dari list yang sudah ada dengan sintaks yang lebih singkat dan efisien. Dengan cara ini, kita bisa melakukan iterasi sekaligus menerapkan kondisi atau operasi tertentu pada setiap elemen. Misalnya, kita dapat menggunakan *list comprehension* untuk memeriksa setiap string dalam sebuah list dan menghasilkan list baru yang berisi nilai boolean (True atau False) yang menunjukkan apakah string tersebut mengandung substring "or".

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if "or" in x]
```

Contoh untuk mengubah semua elementlist menjadi huruf besar.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x.upper() for x in nama]
```

Contoh untuk memeriksa mengambil elemen list yang panjangnya lebih dari 6 huruf.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if len(x) > 6]
```

List Sebagai Parameter Fungsi

Salah satu hal penting yang perlu dipahami saat menggunakan list dalam fungsi adalah bahwa list bersifat *mutable* ketika dikirimkan sebagai argumen. Artinya, perubahan yang dilakukan terhadap list di dalam fungsi akan berdampak

langsung pada list aslinya di luar fungsi. Ini berbeda dengan tipe data seperti integer atau string yang tidak terpengaruh oleh perubahan di dalam fungsi. Karena itu, perancang fungsi perlu berhati-hati agar tidak secara tidak sengaja memodifikasi data asli yang tidak seharusnya berubah.

```
>>> def ubah(data):  
...     data[0] = "anton"  
...  
>>> data = ["a", "b", "c"]  
>>> ubah(data)  
>>> data  
['anton', 'b', 'c']
```

Meski demikian, menggunakan list sebagai parameter fungsi memiliki sejumlah keunggulan, yaitu:

- Fleksibel: fungsi mampu menerima berbagai jenis data yang dikemas dalam list.
- Dapat dipakai ulang: fungsi yang sama bisa diterapkan pada list yang berbeda-beda.
- Memudahkan pembagian tugas: list membantu memecah permasalahan besar menjadi bagian-bagian yang lebih kecil dan terkelola.
- Data lebih terstruktur: list membantu mengorganisir data dengan rapi sebelum diproses oleh fungsi.

Contoh lain untuk menghapus elemen pertama list:

```
>>> def hapus(data):  
...     return data[1:]  
...  
>>> hapus(data)  
['b', 'c']  
>>> data  
['anton', 'b', 'c']  
>>> data = hapus(data)  
>>> data  
['b', 'c']
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
1 def tiga_bilangan_terbaik(lst):
2     unique_num = set(lst)
3     sorted_num = sorted(unique_num, reverse=True)
4     return sorted_num[:3]
5
6 def main():
7     bilangan = []
8     print("Masukkan 3 bilangan satu per satu: ")
9
10    for i in range(3):
11        while True:
12            try:
13                number = int(input(f"Input bilangan ke-{i+1}: "))
14                bilangan.append(number)
15                break
16            except ValueError:
17                print("Input harus berupa bilangan bulat valid, coba lagi.")
18
19    terbaik = tiga_bilangan_terbaik(bilangan)
20    print("3 bilangan terbaik adalah:", terbaik)
21
22 if __name__ == "__main__":
23     main()
```

```
Input bilangan ke-1: 15
Input bilangan ke-2: 17
Input bilangan ke-3: 12
3 bilangan terbaik adalah: [17, 15, 12]
```

Penjelasan:

def tiga_bilangan_terbaik(lst):

- Mendefinisikan fungsi bernama tiga_bilangan_terbaik yang menerima satu parameter.
- Parameter lst berisi daftar angka yang akan diproses.

unique_num = set(lst)

- Mengonversi daftar angka menjadi set untuk menghilangkan nilai-nilai yang berulang.
- Contoh: [8, 9, 9] akan menjadi {8, 9}.

sorted_num = sorted(unique_num, reverse=True)

- Mengurutkan angka-angka unik dari nilai terbesar ke terkecil.
- Contoh: {8, 9} menjadi [9, 8].

return sorted_num [:3]

- Mengembalikan maksimal tiga angka teratas dari hasil pengurutan.
- Jika jumlah elemen kurang dari tiga, semua elemen yang ada dikembalikan.

SOAL 2

```
1 def rata_rata():
2     jumlah = 0
3     count = 0
4
5     while True:
6         user_inp = input("Masukkan bilangan atau ketik 'done' untuk selesai: ")
7
8         if user_inp.lower() == 'done':
9             break
10
11         if user_inp.replace('.', '', 1).isdigit():
12             jumlah += float(user_inp)
13             count += 1
14         else:
15             print("Input tidak valid")
16
17     if count > 0:
18         print(f"Rata-rata bilangan: {jumlah/count}")
19     else:
20         print("Tidak ada bilangan yang dimasukkan")
21
22 rata_rata()
```

```
Masukkan bilangan atau ketik 'done' untuk selesai: 13
Masukkan bilangan atau ketik 'done' untuk selesai: 17
Masukkan bilangan atau ketik 'done' untuk selesai: done
Rata-rata bilangan: 15.0
```

Penjelasan:

jumlah = 0 dan count = 0

- Menyiapkan variabel total untuk menyimpan hasil penjumlahan seluruh angka yang dimasukkan.
- Variabel count bertugas mencatat jumlah angka valid yang telah diinput.

while True:

- Menjalankan perulangan tanpa batas hingga pengguna mengetik 'done'.

user_input = input(...)

- Menerima teks yang diketikkan pengguna, baik berupa angka maupun kata 'done'.

if user_input.lower() == 'done':

- Memeriksa apakah yang diketik adalah 'done' tanpa memedulikan huruf besar atau kecil.
- Jika benar, perulangan dihentikan dengan break.

if user_input.replace('.', '', 1).isdigit():

- Memverifikasi apakah input berupa angka yang valid, termasuk bilangan desimal.
- Satu titik desimal dihapus sementara agar angka seperti "12.5" tetap dikenali sebagai angka.
- Jika valid, angka dikonversi ke tipe float dan ditambahkan ke total, lalu count bertambah satu.

else:

- Menangani input yang bukan angka dan bukan 'done'.
- Menampilkan pesan bahwa input tidak dapat diproses.

if count > 0:

- Setelah perulangan selesai, jika terdapat angka valid yang tercatat, program menghitung rata-rata dengan membagi total dengan count lalu menampilkan hasilnya.

else:

- Jika tidak ada satu pun angka valid yang dimasukkan, program memberitahu bahwa tidak ada data yang dapat dihitung.

SOAL 3

```

1 def mencari_kata_unik(file_path):
2     try:
3         with open(file_path, 'r') as file:
4             artikel = file.read()
5             artikel = artikel.lower()
6             daftar_kata = artikel.split()
7             kata_unik = set(daftar_kata)
8             print("Kata-kata unik yang ditemukan:")
9             for kata in kata_unik:
10                print(kata)
11
12    except FileNotFoundError:
13        print(f"File {file_path} tidak ditemukan")
14    except Exception as e:
15        print(f"Terjadi error: {e}")
16
17 file_path = input("Masukkan path file teks: ")
18 mencari_kata_unik(file_path)

```

```

Masukkan path file teks: text.txt
Kata-kata unik yang ditemukan:
urban
seorang
hunters.
bergenre
"kpop
merupakan
konsep
potensi.
adalah
unik
daya
genre
bernama
yang
ide
fantasi
hunters
grup

```

```

film
juga
tarik
k-pop,
tentang
sebuah
menarik
dan
hunters"
demon
fiksi
k-pop
musikal
penuh
huntr/x
menciptakan
menggabungkan
animasi
dengan
global
netflix

```

Penjelasan:

def mencari_kata_unik(file_path):

- Mendefinisikan fungsi bernama cari_kata_unik dengan satu parameter file_path.
- Parameter ini mewakili lokasi file teks yang akan dibaca dan diproses.

try:

- Menandai blok kode yang akan dipantau kemungkinan terjadinya kesalahan.
- Jika terjadi error saat membuka atau membaca file, program dialihkan ke blok `except`.

with open(file_path, 'r') as file:

- Membuka file dari lokasi yang ditunjuk oleh `file_path` dalam mode baca ('r').
- Penggunaan `with` memastikan file otomatis tertutup setelah selesai diproses tanpa perlu memanggil `file.close()` secara manual.

artikel = file.read()

- Membaca seluruh isi file sekaligus dan menyimpannya ke dalam variabel `artikel` bertipe string.

artikel = artikel.lower()

- Mengubah semua huruf dalam teks menjadi huruf kecil.
- Tujuannya agar kata yang sama tetapi berbeda kapitalisasinya, seperti "Python" dan "python", dianggap sebagai satu kata yang sama.

daftar_kata = artikel.split()

- Memecah isi teks menjadi daftar kata berdasarkan spasi atau karakter pemisah lainnya.
- Hasilnya berupa list yang berisi setiap kata dalam teks.

kata_unik = set(daftar_kata)

- Mengonversi daftar kata menjadi set sehingga setiap kata yang muncul lebih dari sekali hanya disimpan satu kali.

for kata in kata_unik: print(kata)

- Menelusuri setiap kata dalam `kata_unik` dan mencetaknya satu per satu ke layar.
- Karena set tidak memiliki urutan tetap, urutan kata yang ditampilkan bisa berbeda setiap kali program dijalankan.

except FileNotFoundError:

- Menangani situasi di mana file yang diminta tidak ditemukan pada path yang diberikan.
- Menampilkan pesan yang menginformasikan nama file yang tidak berhasil ditemukan.

except Exception as e:

- Menangkap segala jenis kesalahan lain yang mungkin terjadi di luar FileNotFoundError.
- Menampilkan pesan berisi detail error agar pengguna mengetahui sumber masalahnya.

file_path = input(...)

- Meminta pengguna mengetikkan lokasi file teks yang ingin diproses.

mencari_kata_unik(file_path)

- Memanggil fungsi dengan path yang telah dimasukkan agar seluruh proses berjalan.